

Les Principes et l'Utilisation du Modèle Modèle-Vue-Contrôleur (MVC)

1. Introduction

- 1.1. Définition
- 1.2. Origines et utilité du pattern MVC
- 1.3. Avantages et limites

2. Les composants du pattern MVC (exemple PHP)

- 2.1. Le Modèle (Model)
- 2.2. Le Contrôleur (Controller)
- 2.3. La Vue (View)
- 2.3. Le Routeur

3. Exemple JAVA

1. Introduction au MVC

1.1 Définition

Le **Modèle-Vue-Contrôleur (MVC)** est un **modèle (pattern)** qui organise une application en trois composants distincts. L'objectif de ce modèle est de séparer les responsabilités pour faciliter la maintenance, la compréhension et l'évolution du code.

Les 3 composants :

1. **Modèle (Model) :**
 - Contient la logique métier et les données de l'application.
 - Il interagit avec la base de données ou d'autres services pour manipuler les données.
 - Exemples : gestion des utilisateurs, calculs, stockage d'informations.
2. **Vue (View) :**
 - Représente l'interface utilisateur.
 - Affiche les données du modèle et capte les interactions de l'utilisateur.
 - Exemples : interface graphique, pages web.
3. **Contrôleur (Controller) :**
 - Joue le rôle d'intermédiaire entre le modèle et la vue.
 - Reçoit les commandes de l'utilisateur (via la vue), les traduit en actions pour le modèle, puis met à jour la vue en conséquence.

1.2 Origines et utilité du pattern MVC

- **Origines :**

Le MVC a été introduit dans les années 1970 dans le langage **Smalltalk**, utilisé pour concevoir des interfaces graphiques.
- **Utilité :**

Ce modèle est devenu un standard dans le développement logiciel grâce à sa capacité à organiser les applications complexes. Il est omniprésent dans des frameworks modernes comme :

 - **Spring MVC** en Java
 - **Ruby on Rails**
 - **Django** pour Python
 - **Angular** ou **React** côté web.

1.3 Avantages et limites

Avantages :

1. **Séparation des préoccupations** : Chaque composant a un rôle bien défini, ce qui rend le code plus lisible.
2. **Réutilisabilité** : La vue peut être modifiée ou remplacée sans affecter la logique métier.
3. **Testabilité** : Les composants peuvent être testés séparément.
4. **Évolution facilitée** : Ajout de nouvelles fonctionnalités plus simple.

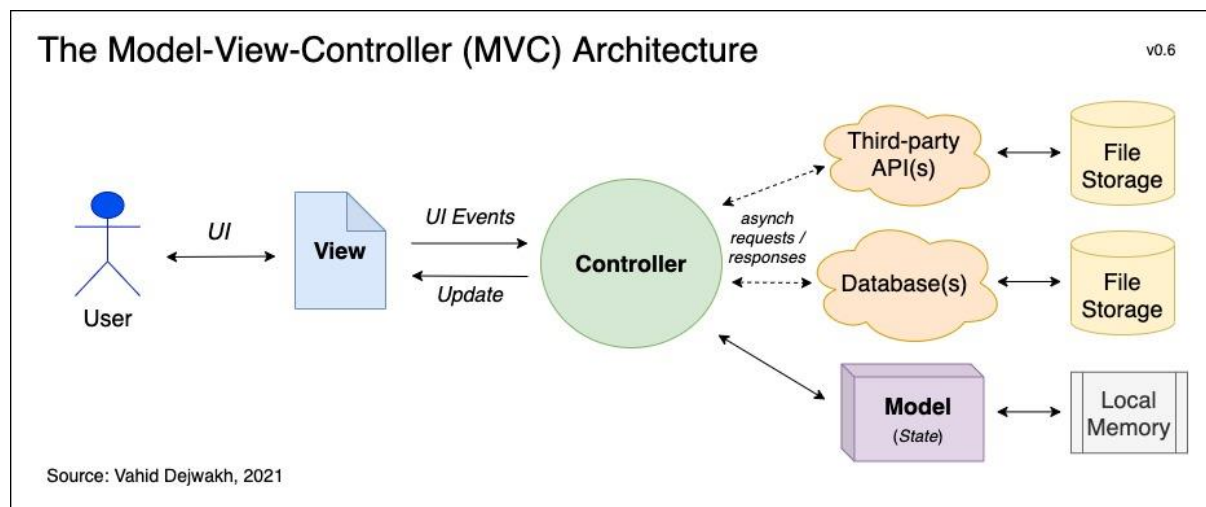
Limites :

1. **Complexité accrue** : Implémenter le pattern peut être lourd pour de petites applications.
2. **Apprentissage initial** : Il nécessite une bonne compréhension des interactions entre les composants.
3. **Couplage indirect** : Les dépendances entre les composants peuvent devenir compliquées si elles ne sont pas bien définies.

Exemple

Prenons l'exemple d'un restaurant :

1. **Modèle** : La cuisine.
 - Prépare les plats (données/logiciel métier).
 - Ne se soucie pas de savoir comment les plats seront servis ou qui les commande.
2. **Vue** : Le serveur.
 - Présente les plats au client et prend les commandes.
 - Ne sait pas comment les plats sont préparés.
3. **Contrôleur** : Le responsable de salle.
 - Organise les commandes entre les serveurs et la cuisine.
 - Vérifie que tout est conforme et bien synchronisé.



2. Les composants du MVC (PHP)

Exemple de gestion de tâches

2.1 Model (Task.php)

- Représente les données et la logique métier.
- Ne contient aucune logique liée à l'interface utilisateur.

```
<?php
// app/models/Task.php
class Task {
    private $db;

    public function __construct() {
        $this->db = new PDO('mysql:host=localhost;dbname=mvc_example', 'root', '');
    }

    public function getAllTasks() {
        $query = $this->db->prepare("SELECT * FROM tasks");
        $query->execute();
        return $query->fetchAll(PDO::FETCH_ASSOC);
    }

    public function addTask($taskName) {
        $query = $this->db->prepare("INSERT INTO tasks (name) VALUES (:name)");
        $query->bindParam(':name', $taskName);
        $query->execute();
    }

    public function deleteTask($taskId) {
        $query = $this->db->prepare("DELETE FROM tasks WHERE id = :id");
        $query->bindParam(':id', $taskId);
        $query->execute();
    }
}
```

2.2 Contrôleur (TaskController.php)

- Interagit avec la vue pour réagir aux actions de l'utilisateur.
- Met à jour le modèle en conséquence.

```
require_once '../app/models/Task.php';

class TaskController {
    private $taskModel;

    public function __construct() {
        $this->taskModel = new Task();
    }

    public function index() {
        $tasks = $this->taskModel->getAllTasks();
        require '../app/views/taskList.php';
    }

    public function addTask($taskName) {
        $this->taskModel->addTask($taskName);
        header('Location: /');
    }

    public function deleteTask($taskId) {
        $this->taskModel->deleteTask($taskId);
        header('Location: /');
    }
}
```

2.3 View (taskList.php)

- Responsable de la présentation des données au format voulu.
- Reçoit les données du Modèle mais ne les modifie pas.

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Liste des Tâches</title>
</head>
<body>
    <h1>Liste des Tâches</h1>
    <ul>
        <?php foreach ($tasks as $task): ?>
            <li><?php echo htmlspecialchars($task['name']); ?>
                <a href="/delete?id=<?php echo $task['id']; ?>">Supprimer</a>
            </li>
        <?php endforeach; ?>
    </ul>
    <form method="POST" action="/add">
        <input type="text" name="taskName" placeholder="Nouvelle tâche">
        <button type="submit">Ajouter</button>
    </form>
</body>
</html>
```

2.4 Routeur (index.php)

```
<?php
require_once '../app/controllers/TaskController.php';

$controller = new TaskController();

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['taskName'])) {
    $controller->addTask($_POST['taskName']);
} elseif (isset($_GET['id'])) {
    $controller->deleteTask($_GET['id']);
} else {
    $controller->index();
}
```

3. Les composants du MVC (JAVA)

Exemple de gestions de tâches.

3.1 Le Modèle (Model)

```
public class Task {
    private String name;
    private boolean completed;

    public Task(String name) {
        this.name = name;
        this.completed = false;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public boolean isCompleted() {
        return completed;
    }

    public void setCompleted(boolean completed) {
        this.completed = completed;
    }
}
```

3.2 La Vue (View)

```
public class TaskView {
    public void displayTask(String taskName, boolean completed) {
        System.out.println("Tâche : " + taskName + " | État : " +
            (completed ? "Terminée" : "Non terminée"));
    }
}
```

3.3 Le Contrôleur (Controller)

```
public class TaskController {
    private Task model;
    private TaskView view;

    public TaskController(Task model, TaskView view) {
        this.model = model;
        this.view = view;
    }

    public void setTaskName(String name) {
        model.setName(name);
    }

    public void setTaskCompletion(boolean completed) {
        model.setCompleted(completed);
    }

    public void updateView() {
        view.displayTask(model.getName(), model.isCompleted());
    }
}
```

3.4 Main

```
public class Main {  
    public static void main(String[] args) {  
        // Création du modèle  
        Task task = new Task("Apprendre le MVC");  
  
        // Création de la vue  
        TaskView view = new TaskView();  
  
        // Création du contrôleur  
        TaskController controller = new TaskController(task, view);  
  
        // Mise à jour et affichage  
        controller.updateView();  
  
        // Modification via le contrôleur  
        controller.setTaskCompletion(true);  
        controller.updateView();  
    }  
}
```

3.5 Résultat

Tâche : Apprendre le MVC | État : Non terminée
Tâche : Apprendre le MVC | État : Terminée